HashiCorp Certified: Terraform Associate（004）　目標 7a〜7c　#14

インポート・stateコマンド・ログ

import ブロックと手順、インポート ID、-generate-config-out による設定の生成、terraform import コマンド、terraform state の各サブコマンド（list・show・pull・mv・rm・push・replace-provider）とバックアップ、TF_LOG・TF_LOG_PATH・TF_LOG_CORE・TF_LOG_PROVIDER

各ページの下に、根拠にした資料（HashiCorp の Exam Content List（004）、Terraform ドキュメント v1.12.x の Import existing resources・import block reference・Generate configuration・Command: import・Import existing resources overview・terraform state commands・Command: state mv・state rm・state push・state replace-provider・Enable Terraform logs・Environment Variables）を書いた。英語の原文は『』で引用した

第1章

インポート

外で作ったものを、Terraform の管理に入れる。

import ブロックの手順

```
# imports.tf
import {
  to = aws_s3_bucket.this      # 取りこみ先のアドレス
  id = "example-bucket"        # プロバイダーでの ID
}
# main.tf
resource "aws_s3_bucket" "this" {
  bucket = "example-bucket"
}
```

id は plan のときに分かる文字列。identity（属性の組）とは同時に書けない。for_each・provider も書ける。



resource がないと、次の計画で消そうとする。

出典：Import existing resources／import block reference

設定の生成とインポート ID

```
$ terraform plan -generate-config-out="generated.tf"
```

決まり
import ブロックの to が設定にないときに HCL を書き出す
まだないファイルのパス（あるとエラー）
『best guess』なので見なおしてから VCS へ
v1.5 からの実験的な機能

リソース	ID の例
EC2	i-abcd1234
Route53 ゾーン	Z12ABC4UGMOZ2N
S3 バケット	バケット名

ID の形と、インポートできるかどうかは Registry の各リソースのページの下で確かめる。

1 つの実物は 1 つのアドレスだけに取りこむ。

出典：Generate configuration／Command: import

import ブロックと terraform import

	import ブロック	terraform import コマンド
書く場所	設定（imports.tf など）	CLI：terraform import ADDRESS ID
確かめ方	plan で見てから apply	すぐステートに入る
一度に	複数（for_each も可）	1 つ
設定の生成	-generate-config-out	しない（resource を先に手で書く）
CI/CD	組みこみやすい	向かない
HCP Terraform	実行の中で動く	ローカルで動く（ワークスペースの変数をローカルで用意）

どちらも、残りのライフサイクルを管理するには resource ブロックが必要。

出典：Command: import／Import existing resources overview

第2章

state コマンド

ステートを直接いじらずに、見る・直す。

terraform state のサブコマンド

コマンド	何をするか	ステート
state list	リソースのアドレスを一覧	変えない
state show ADDRESS	1 つのリソースの属性	変えない
state pull	ステート（JSON）を標準出力へ	変えない
state mv SRC DST	アドレスを変える（同じ型だけ。今は moved）	変える
state rm ADDRESS	管理をやめる。実物は残る（今は removed）	変える
state push PATH	ローカルのステートでバックエンドを上書き	変える
state replace-provider	プロバイダーのソースを付けかえ	変える

ステートを変えるものは必ずバックアップを書き、止められない：『backups for state modification can not be disabled』

出典：terraform state commands

mv・rm・push の注意点

```
$ terraform state mv -dry-run aws_instance.a aws_instance.b
$ terraform state mv aws_instance.a aws_instance.b
$ terraform state rm aws_instance.web
$ terraform state push ./fixed.tfstate
$ terraform state replace-provider \
    hashicorp/aws registry.acme.corp/acme/aws
```

-dry-run（mv・rm）：変えずに対象だけ表示。

state mv
型は変えられない。チームでは設定の変更との間にだれも変更しないよう連絡

state rm
設定が残っていれば次の plan で「新しく作る」

state push
リモートを手で直すときだけ。lineage がちがう・serial が小さいと拒否（-force は危険）

リモートステートでも同じように使える（毎回ネットワークの往復）。

出典：Command: state mv／state rm／state push／state replace-provider

第3章

ログ

エラーだけで分からないときに。

TF_LOG とその仲間

```
$ export TF_LOG=TRACE
$ export TF_LOG_PATH=./terraform.log   # 追記
$ terraform apply

$ export TF_LOG_PROVIDER=DEBUG         # プロバイダーだけ
$ unset TF_LOG                         # または TF_LOG=off
```

くわしい ← TRACE・DEBUG・INFO・WARN・ERROR → くわしくない

JSON：TRACE 以上を JSON で（安定した形式ではない）

変数	役割
TF_LOG	有効にする。出力は stderr。どんな値でも有効
TF_LOG_PATH	ファイルに追記（TF_LOG も必要）
TF_LOG_CORE	Terraform 本体だけ
TF_LOG_PROVIDER	プロバイダーのプラグインだけ

『even when TF_LOG_PATH is set, TF_LOG must be set』

使う場面：予期しない動きのデバッグ、バグ報告。ログに機密が入りうる。

出典：Enable Terraform logs／Environment Variables

本番での問われ方

ねらい	見るところ
import ブロック（7a）	to・id、手順、plan で確かめる、resource が必要、-generate-config-out は新しいファイル
import コマンド（7a）	ADDRESS ID、設定は作らない、1 つずつ、HCP ではローカルで動く
state（7b）	list・show・pull は読むだけ、mv・rm・push・replace-provider、バックアップは止められない、-dry-run
ログ（7c）	TF_LOG のレベルの順、stderr、TF_LOG_PATH だけでは出ない、CORE・PROVIDER、JSON、off

出典：Exam Content List（004）7a〜7c／本資料の問題演習（全 30 問）